

CSCI3170

Introduction to Database Systems

6-sql_inter

Slides: Michael Yu

Aggregate Functions Examples

- Find the average salary of instructors in the Computer Science department

```
– select avg (salary)
  from instructor
  where dept_name= 'Comp. Sci.';
```

- Find the total number of instructors who teach a course in the Spring 2018 semester

```
– select count (distinct ID)
  from teaches
  where semester = 'Spring' and year = 2018;
```

- Find the number of tuples in the *course* relation

```
– select count (*)
  from course;
```

Recap: Aggregate Functions - Group By

- Find the average salary of instructors in each department
 - `select dept_name, avg (salary) as avg_salary`
`from instructor`
`group by dept_name;`

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

<i>dept_name</i>	<i>avg_salary</i>
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

Other Aggregate Operators

- SQL allows use of arithmetic expressions
 - SQL supports five aggregate operations, which can be applied on any column of a relation.
 - **COUNT, SUM, AVG, MAX, MIN**
 - Note: Most aggregation operators generally apply to numeric values in one column at a time
- **Example:** COUNT() with diff argument used:
 - COUNT (*)
 - no. of rows in a table (including NULL)
 - COUNT(columnname)
 - no. of all non-null values in column
 - COUNT(DISTINCT columnname)
 - no. of unique, non-null values in column

On Aggregate Operators

- **COUNT (*)** returns no. of rows in a table (including NULL)
- **Example:** How many different beers are there?

– **SELECT** COUNT (*)
FROM Beers;

COUNT(*)

18

- Find number of tuples in the *course* relation

– **select count** (*)
from *course*;

Beers:

Name	Manf
80/-	Caledonian
Bigfoot Barley Wine	Sierra Nevada
Burraborang Bock	George IV Inn
Crown Lager	Carlton
Fosters Lager	Carlton
Invalid Stout	Carlton
Melbourne Bitter	Carlton
New	Toohey's
Old	Toohey's
Old Admiral	Lord Nelson
Pale Ale	Sierra Nevada
Premium Lager	Cascade
Red	Toohey's
Sheaf Stout	Toohey's
Sparkling Ale	Cooper's
Stout	Cooper's
Three Sheets	Lord Nelson
Victoria Bitter	Carlton

course_id	title	dept_name	credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

On Aggregate Operators

- **Example:** Return no. of bars that sells beer?

```
– SELECT COUNT(DISTINCT bar)
FROM Sells;
```

COUNT(DISTINCT BAR)

6

- **Example:** Return average price of *New*?

```
– SELECT AVG(price)
FROM Sells
WHERE beer = 'New';
```

AVG(PRICE)

2.3875

Sells:

Bar	Beer	Price
Australia Hotel	Burraborang Bock	3.5
Coogee Bay Hotel	New	2.25
Coogee Bay Hotel	Old	2.5
Coogee Bay Hotel	Sparkling Ale	2.8
Coogee Bay Hotel	Victoria Bitter	2.3
Lord Nelson	Three Sheets	3.75
Lord Nelson	Old Admiral	3.75
Marble Bar	New	2.8
Marble Bar	Old	2.8
Marble Bar	Victoria Bitter	2.8
Regent Hotel	New	2.2
Regent Hotel	Victoria Bitter	2.2
Royal Hotel	New	2.3
Royal Hotel	Old	2.3
Royal Hotel	Victoria Bitter	2.3

Same as AVG (DISTINCT price) ?

We can force 'set semantics' to aggregations using DISTINCT

How to Include NULL Values as Zero?

```
SELECT AVG(COALESCE(column_name, 0)) FROM table_name;
```

Check Understanding Check

- Find average age of all sailors?
- Find average age of sailors with rating of 10
- Count number of Sailors
- Count number of different sailor names

sailors

<u>sid</u>	sname	rating	age
------------	-------	--------	-----

Boats

<u>bid</u>	bname	color
------------	-------	-------

Reserves

<u>sid</u>	<u>bid</u>	day
------------	------------	-----

Null Values on Aggregations**

- All aggregate operations except **count(*)** ignore tuples with null values on the aggregated attributes
- What if collection has only null values?
 - Count(columnname) returns 0
 - Other aggregates (e.g., Average, Max, Min, Sum) returns null
- i.e., Context total of all salaries
`select sum (salary) from instructor`
 - Ignores null amounts
 - Result is *null* if there is no non-null amount

M: You will see this in future labs!

Nested Subqueries

- SQL provides a mechanism for nesting of subqueries
 - i.e., **select-from-where** expression nested in another query.
- The nesting can be done in the following SQL query

```
select A1, A2, ..., An  
from r1, r2, ..., rm  
where P
```

May be deeply nested!

Common use:

1. set membership
2. set comparisons
3. set cardinality

as follows:

- **From clause:** r_i can be replaced by any valid subquery
- **Where clause:** P can be replaced with an expression of the form:
$$B <\text{operation}> (\text{subquery})$$

B is an attribute and $<\text{operation}>$ to be defined later.
- **Select clause:**
 A_i can be replaced by a subquery that generates a single value.

Basic Subquery: Scalar Subqueries

Motivating Example

- **Query:** Return bars selling *New* at same price as Coogee Bay Hotel charges for the beer *VB*

Sells:

Bar	Beer	Price
Australia Hotel	Burraborang Bock	3.5
Coogee Bay Hotel	New	2.25
Coogee Bay Hotel	Old	2.5
Coogee Bay Hotel	Sparkling Ale	2.8
Coogee Bay Hotel	Victoria Bitter	2.3
Lord Nelson	Three Sheets	3.75
Lord Nelson	Old Admiral	3.75
Marble Bar	New	2.8
Marble Bar	Old	2.8
Marble Bar	Victoria Bitter	2.8
Regent Hotel	New	2.2
Regent Hotel	Victoria Bitter	2.2
Royal Hotel	New	2.3
Royal Hotel	Old	2.3
Royal Hotel	Victoria Bitter	2.3

- Option 1 (without nested-query)
 - ```
SELECT b2.bar
FROM Sells b1, Sells b2
WHERE b1.beer = 'Victoria Bitter'
AND b1.bar = 'Coogee Bay Hotel'
AND b1.price = b2.price
AND b2.beer = 'New';
```

# Scalar Subquery

- **Query:** Return bars selling *New* at same price as Coogee Bay Hotel charges for the beer *VB*

- Option 2 (with nested-query)

```
– SELECT bar
 FROM Sells
 WHERE beer = 'New'
 AND price = (
 SELECT price
 FROM Sells
 WHERE bar = 'Coogee Bay Hotel'
 AND beer = 'Victoria Bitter') ;
```

## Sells:

| Bar              | Beer             | Price |
|------------------|------------------|-------|
| Australia Hotel  | Burraborang Bock | 3.5   |
| Coogee Bay Hotel | New              | 2.25  |
| Coogee Bay Hotel | Old              | 2.5   |
| Coogee Bay Hotel | Sparkling Ale    | 2.8   |
| Coogee Bay Hotel | Victoria Bitter  | 2.3   |
| Lord Nelson      | Three Sheets     | 3.75  |
| Lord Nelson      | Old Admiral      | 3.75  |
| Marble Bar       | New              | 2.8   |
| Marble Bar       | Old              | 2.8   |
| Marble Bar       | Victoria Bitter  | 2.8   |
| Regent Hotel     | New              | 2.2   |
| Regent Hotel     | Victoria Bitter  | 2.2   |
| Royal Hotel      | New              | 2.3   |
| Royal Hotel      | Old              | 2.3   |
| Royal Hotel      | Victoria Bitter  | 2.3   |

- Scalar subquery is one which is used where a single value is expected
- Runtime error if subquery returns more than one result tuple

# Scalar Subquery (Sample Use-case)

- Return name and age of oldest sailor?

– */\* erroneous query \*/*

```
SELECT sname, MAX (age)
FROM Sailors;
```

*Attributes in **select** clause outside of aggregate functions must appear in **group by** list. (Another issue query is **select dept\_name, ID, avg (salary) from instructor group by dept\_name;**)*

– 

```
SELECT sname, age
FROM Sailors
WHERE age = (SELECT MAX (age)
 FROM Sailors);
```

|            |                                                                                      |            |            |            |     |
|------------|--------------------------------------------------------------------------------------|------------|------------|------------|-----|
| sailors    | <table><tr><td><u>sid</u></td><td>sname</td><td>rating</td><td>age</td></tr></table> | <u>sid</u> | sname      | rating     | age |
| <u>sid</u> | sname                                                                                | rating     | age        |            |     |
| Boats      | <table><tr><td><u>bid</u></td><td>bname</td><td>color</td></tr></table>              | <u>bid</u> | bname      | color      |     |
| <u>bid</u> | bname                                                                                | color      |            |            |     |
| Reserves   | <table><tr><td><u>sid</u></td><td><u>bid</u></td><td><u>day</u></td></tr></table>    | <u>sid</u> | <u>bid</u> | <u>day</u> |     |
| <u>sid</u> | <u>bid</u>                                                                           | <u>day</u> |            |            |     |

# Scalar Subquery (Sample Use-case)

- Comprehension Exercise

```
– SELECT sname
 FROM Sailors
 WHERE age > (SELECT MAX (age)
 FROM Sailors
 WHERE rating = 10);
```

|            |                                                                                      |            |            |            |     |
|------------|--------------------------------------------------------------------------------------|------------|------------|------------|-----|
| sailors    | <table><tr><td><u>sid</u></td><td>sname</td><td>rating</td><td>age</td></tr></table> | <u>sid</u> | sname      | rating     | age |
| <u>sid</u> | sname                                                                                | rating     | age        |            |     |
| Boats      | <table><tr><td><u>bid</u></td><td>bname</td><td>color</td></tr></table>              | <u>bid</u> | bname      | color      |     |
| <u>bid</u> | bname                                                                                | color      |            |            |     |
| Reserves   | <table><tr><td><u>sid</u></td><td><u>bid</u></td><td><u>day</u></td></tr></table>    | <u>sid</u> | <u>bid</u> | <u>day</u> |     |
| <u>sid</u> | <u>bid</u>                                                                           | <u>day</u> |            |            |     |

## Subquery: Set Membership

# IN Operator

- Allows you to check for membership in provided set of values.
- Find the names of boats which are red, blue, or green:

```
- SELECT bname
 FROM Boats
 WHERE color IN ('red', 'blue', 'green');
```

|            |                                                                                      |            |            |            |     |
|------------|--------------------------------------------------------------------------------------|------------|------------|------------|-----|
| sailors    | <table><tr><td><u>sid</u></td><td>sname</td><td>rating</td><td>age</td></tr></table> | <u>sid</u> | sname      | rating     | age |
| <u>sid</u> | sname                                                                                | rating     | age        |            |     |
| Boats      | <table><tr><td><u>bid</u></td><td>bname</td><td>color</td></tr></table>              | <u>bid</u> | bname      | color      |     |
| <u>bid</u> | bname                                                                                | color      |            |            |     |
| Reserves   | <table><tr><td><u>sid</u></td><td><u>bid</u></td><td><u>day</u></td></tr></table>    | <u>sid</u> | <u>bid</u> | <u>day</u> |     |
| <u>sid</u> | <u>bid</u>                                                                           | <u>day</u> |            |            |     |

# IN Operator

- Allows you to check for membership in SQL query result set.
- Find names of sailors who've reserved boat #103:

```
- SELECT sname
 FROM Sailors
 WHERE sid IN(SELECT sid
 FROM Reserves
 WHERE bid=103);
```

- Note: to find sailors who've *not* reserved #103, use **NOT IN**.
- *This query can be implemented easily without nested structure*

|            |                                                                                      |            |            |            |     |
|------------|--------------------------------------------------------------------------------------|------------|------------|------------|-----|
| sailors    | <table><tr><td><u>sid</u></td><td>sname</td><td>rating</td><td>age</td></tr></table> | <u>sid</u> | sname      | rating     | age |
| <u>sid</u> | sname                                                                                | rating     | age        |            |     |
| Boats      | <table><tr><td><u>bid</u></td><td>bname</td><td>color</td></tr></table>              | <u>bid</u> | bname      | color      |     |
| <u>bid</u> | bname                                                                                | color      |            |            |     |
| Reserves   | <table><tr><td><u>sid</u></td><td><u>bid</u></td><td><u>day</u></td></tr></table>    | <u>sid</u> | <u>bid</u> | <u>day</u> |     |
| <u>sid</u> | <u>bid</u>                                                                           | <u>day</u> |            |            |     |

# Set Membership Example

- Find name and brewers of beers that John likes.

## Likes:

| Drinker | Beer                |
|---------|---------------------|
| Adam    | Crown Lager         |
| Adam    | Fosters Lager       |
| Adam    | New                 |
| Gernot  | Premium Lager       |
| Gernot  | Sparkling Ale       |
| John    | 80/-                |
| John    | Bigfoot Barley Wine |
| John    | Pale Ale            |
| John    | Three Sheets        |
| Justin  | Sparkling Ale       |
| Justin  | Victoria Bitter     |

## Beers:

| Name                | Manf                                                                                              |
|---------------------|---------------------------------------------------------------------------------------------------|
| 80/-                | Caledonian     |
| Bigfoot Barley Wine | Sierra Nevada  |
| Burraborang Bock    | George IV Inn                                                                                     |
| Crown Lager         | Carlton                                                                                           |
| Fosters Lager       | Carlton                                                                                           |
| Invalid Stout       | Carlton                                                                                           |
| Melbourne Bitter    | Carlton                                                                                           |
| New                 | Toohey's                                                                                          |
| Old                 | Toohey's                                                                                          |
| Old Admiral         | Lord Nelson                                                                                       |
| Pale Ale            | Sierra Nevada  |
| Premium Lager       | Cascade                                                                                           |
| Red                 | Toohey's                                                                                          |
| Sheaf Stout         | Toohey's                                                                                          |
| Sparkling Ale       | Cooper's                                                                                          |
| Stout               | Cooper's                                                                                          |
| Three Sheets        | Lord Nelson  |
| Victoria Bitter     | Carlton                                                                                           |

# Set Membership Example

- Find name and brewers of beers that John likes.

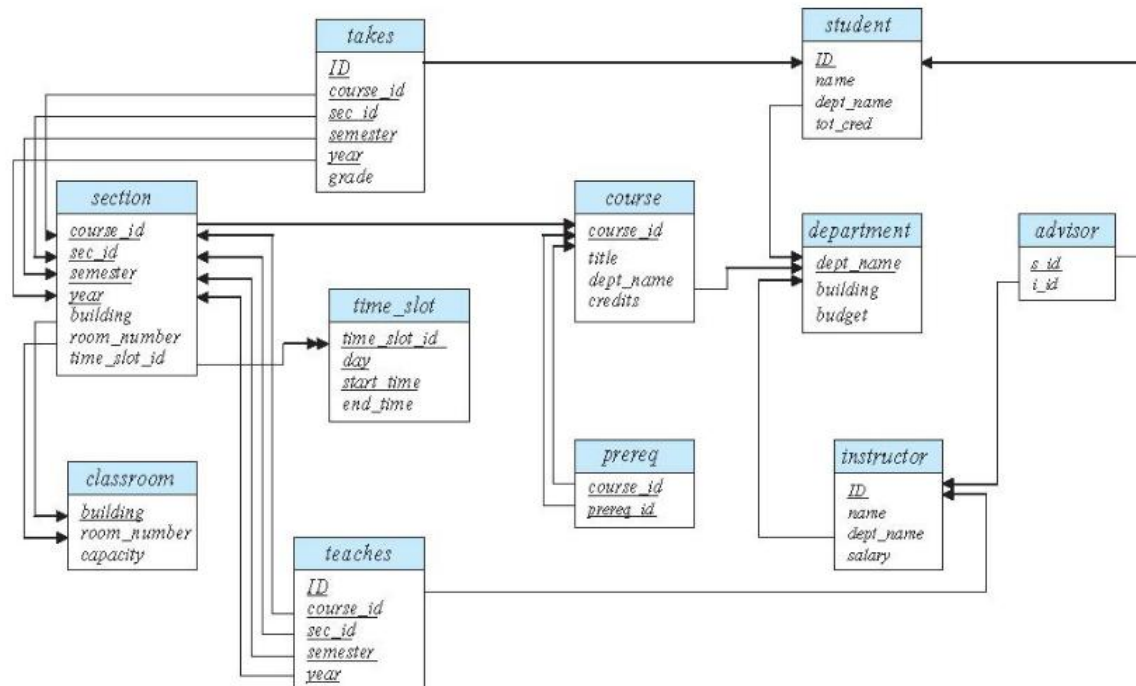
```
– SELECT *
 FROM Beers
 WHERE name IN (SELECT beer
 FROM Likes
 WHERE drinker = 'John');
```

| NAME                | MANF          |
|---------------------|---------------|
| 80/-                | Caledonian    |
| Bigfoot Barley Wine | Sierra Nevada |
| Pale Ale            | Sierra Nevada |
| Three Sheets        | Lord Nelson   |

# Check Understanding

What does this do?

```
select count (distinct ID)
from takes
where (course_id, sec_id, semester, year) in (
 select course_id, sec_id, semester, year
 from teaches
 where teaches.ID = '10101');
```



## Subquery: Set Comparison

# \*\*Set-comparison Operators

- Also available: *op ANY, op ALL, op SOME*
  - Where op an arithmetic comparison: >, <, =, >=, <=, <>.
  - **SOME** just a synonym for **ANY**

# Set Comparison Op - Some/ Any

- Find names of instructors with salary greater than that of some (at least one) instructor in the Physics department

```
select distinct T.name
from instructor as T, instructor as S
where T.salary > S.salary
and S.dept name = 'Physics';
```

- Same query using > **some** clause

```
select name
from instructor
where salary > some (
 select salary
 from instructor
 where dept name = 'Physics');
```

| ID    | name       | dept_name  | salary |
|-------|------------|------------|--------|
| 22222 | Einstein   | Physics    | 95000  |
| 12121 | Wu         | Finance    | 90000  |
| 32343 | El Said    | History    | 60000  |
| 45565 | Katz       | Comp. Sci. | 75000  |
| 98345 | Kim        | Elec. Eng. | 80000  |
| 76766 | Crick      | Biology    | 72000  |
| 10101 | Srinivasan | Comp. Sci. | 65000  |
| 58583 | Califieri  | History    | 62000  |
| 83821 | Brandt     | Comp. Sci. | 92000  |
| 15151 | Mozart     | Music      | 40000  |
| 33456 | Gold       | Physics    | 87000  |
| 76543 | Singh      | Finance    | 80000  |

(a) The *instructor* table

**SOME is identical to ANY**

# Set Comparison Op - Some/ Any

```
select name
from instructor
where salary > some (
 select salary
 from instructor
 where dept name = 'Physics');
```

(= some)  $\equiv$  in  
However, ( $\neq$  some)  $\neq$  not in

(5 < some 

|   |
|---|
| 0 |
| 5 |
| 6 |

) = true

(5 < some 

|   |
|---|
| 0 |
| 5 |

) = false

(5 = some 

|   |
|---|
| 0 |
| 5 |

) = true

(5  $\neq$  some 

|   |
|---|
| 0 |
| 5 |

) = true (since  $0 \neq 5$ )

| ID    | name       | dept_name  | salary |
|-------|------------|------------|--------|
| 22222 | Einstein   | Physics    | 95000  |
| 12121 | Wu         | Finance    | 90000  |
| 32343 | El Said    | History    | 60000  |
| 45565 | Katz       | Comp. Sci. | 75000  |
| 98345 | Kim        | Elec. Eng. | 80000  |
| 76766 | Crick      | Biology    | 72000  |
| 10101 | Srinivasan | Comp. Sci. | 65000  |
| 58583 | Califieri  | History    | 62000  |
| 83821 | Brandt     | Comp. Sci. | 92000  |
| 15151 | Mozart     | Music      | 40000  |
| 33456 | Gold       | Physics    | 87000  |
| 76543 | Singh      | Finance    | 80000  |

(a) The *instructor* table

**SOME is identical to ANY**

# SOME is identical to ANY

- Find sailors whose rating is greater than that of some sailor called Horatio

```
- SELECT *
 FROM Sailors
 WHERE rating > ANY (
 SELECT rating
 FROM Sailors
 WHERE sname='Horatio');
```

SOME is identical to ANY

# New Sample Query

- Find names of instructors whose salary is greater than salary of all instructors in Physics department.

| <i>ID</i> | <i>name</i> | <i>dept_name</i> | <i>salary</i> |
|-----------|-------------|------------------|---------------|
| 22222     | Einstein    | Physics          | 95000         |
| 12121     | Wu          | Finance          | 90000         |
| 32343     | El Said     | History          | 60000         |
| 45565     | Katz        | Comp. Sci.       | 75000         |
| 98345     | Kim         | Elec. Eng.       | 80000         |
| 76766     | Crick       | Biology          | 72000         |
| 10101     | Srinivasan  | Comp. Sci.       | 65000         |
| 58583     | Califieri   | History          | 62000         |
| 83821     | Brandt      | Comp. Sci.       | 92000         |
| 15151     | Mozart      | Music            | 40000         |
| 33456     | Gold        | Physics          | 87000         |
| 76543     | Singh       | Finance          | 80000         |

(a) The *instructor* table

# Sample Solution using ALL

```
select name
from instructor
where salary > all (select salary
 from instructor
 where
 dept name = 'Physics');
```

(**≠ all**) ≡ **not in**  
 However, (**= all**) **≠ in**

(5 < all 

|   |
|---|
| 0 |
| 5 |
| 6 |

) = false

(5 < all 

|    |
|----|
| 6  |
| 10 |

) = true

(5 = all 

|   |
|---|
| 4 |
| 5 |

) = false

(5 ≠ all 

|   |
|---|
| 4 |
| 6 |

) = true (since 5 ≠ 4 and 5 ≠ 6)

| ID    | name       | dept_name  | salary |
|-------|------------|------------|--------|
| 22222 | Einstein   | Physics    | 95000  |
| 12121 | Wu         | Finance    | 90000  |
| 32343 | El Said    | History    | 60000  |
| 45565 | Katz       | Comp. Sci. | 75000  |
| 98345 | Kim        | Elec. Eng. | 80000  |
| 76766 | Crick      | Biology    | 72000  |
| 10101 | Srinivasan | Comp. Sci. | 65000  |
| 58583 | Califieri  | History    | 62000  |
| 83821 | Brandt     | Comp. Sci. | 92000  |
| 15151 | Mozart     | Music      | 40000  |
| 33456 | Gold       | Physics    | 87000  |
| 76543 | Singh      | Finance    | 80000  |

(a) The *instructor* table

# Set-comparison: ALL

- Universal quantifier
- Example: Find the beers sold for the highest price.

```
- SELECT beer
 FROM Sells
 WHERE price >= ALL (
 SELECT price
 FROM sells);
```

BEER

-----  
Three Sheets  
Old Admiral

# Example (1) ALL

- Find sailors whose rating is better than every sailor called Horatio.

```
- SELECT *
 FROM Sailors
 WHERE rating > ALL (
 SELECT rating
 FROM Sailors
 WHERE sname='Horatio');
```

- Find the sailors with the highest rating?

```
- SELECT *
 FROM Sailors
 WHERE rating >= ALL (
 SELECT rating
 FROM Sailors);
```

|            |                                                                                      |            |            |            |     |
|------------|--------------------------------------------------------------------------------------|------------|------------|------------|-----|
| sailors    | <table><tr><td><u>sid</u></td><td>sname</td><td>rating</td><td>age</td></tr></table> | <u>sid</u> | sname      | rating     | age |
| <u>sid</u> | sname                                                                                | rating     | age        |            |     |
| Boats      | <table><tr><td><u>bid</u></td><td>bname</td><td>color</td></tr></table>              | <u>bid</u> | bname      | color      |     |
| <u>bid</u> | bname                                                                                | color      |            |            |     |
| Reserves   | <table><tr><td><u>sid</u></td><td><u>bid</u></td><td><u>day</u></td></tr></table>    | <u>sid</u> | <u>bid</u> | <u>day</u> |     |
| <u>sid</u> | <u>bid</u>                                                                           | <u>day</u> |            |            |     |

# Foreword: Notice that

In all previous examples:

inner subquery completely independent of outer query.

In general...

inner subquery could depend on row currently being  
examined in outer query

# Correlated Nested Queries

- In the previous example, the inner subquery has been completely independent of the outer query.
- In general, the inner subquery could depend on the row currently being examined in the outer query.

# Set-comparison: EXISTS

- Some manufacturers only make one beer.
  - Find the beers made by manufactures that make only one beer.

## Beers:

| Name                | Manf          |
|---------------------|---------------|
| 80/-                | Caledonian    |
| Bigfoot Barley Wine | Sierra Nevada |
| Burraborang Bock    | George IV Inn |
| Crown Lager         | Carlton       |
| Fosters Lager       | Carlton       |
| Invalid Stout       | Carlton       |
| Melbourne Bitter    | Carlton       |
| New                 | Toohey's      |
| Old                 | Toohey's      |
| Old Admiral         | Lord Nelson   |
| Pale Ale            | Sierra Nevada |
| Premium Lager       | Cascade       |
| Red                 | Toohey's      |
| Sheaf Stout         | Toohey's      |
| Sparkling Ale       | Cooper's      |
| Stout               | Cooper's      |
| Three Sheets        | Lord Nelson   |
| Victoria Bitter     | Carlton       |

# Set-comparison: EXISTS

- EXISTS:
  - EXISTS returns true iff relation is non-empty
  - *(In contrast: IN returns true iff membership)*
- Example: Find the beers made uniquely by their manufacturer.

```
– SELECT name
 FROM Beers b1
 WHERE NOT EXISTS (
 SELECT *
 FROM Beers
 WHERE manf = b1.manf
 AND name != b1.name);
```

NAME

-----

80/-

Burraborang Bock  
Premium Lager

# Correlation Variables

- Find all courses taught in both the Fall 2009 semester and in the Spring 2010 semester. Recall the `section` relation is (course\_id, sec\_id, semester, year, building, room\_no, time\_slot\_id)

```
- select course_id
 from section as S
 where semester = 'Fall'
 and year = 2009
 and exists (
 select *
 from section as T
 where semester = 'Spring'
 and year= 2010
 and T.course_id = S.course_id);
```

# Not Exists Use Case - Division

- Query Return students who have taken all courses offered in the Biology department.

**select distinct** *S.ID*, *S.name*

**from** *student* **as** *S*

**where not exists** (**(select** *course\_id*  
**from** *course*  
**where** *dept\_name* = 'Biology')  
**except**  
**(select** *T.course\_id*  
**from** *takes* **as** *T*  
**where** *S.ID* = *T.ID*));

student

| ID | name |
|----|------|
| 10 | A    |
| 20 | B    |

takes

| ID | course_id |
|----|-----------|
| 10 | 100       |
| 10 | 200       |
| 10 | 300       |
| 20 | 200       |

course

| course_id | dept_name |
|-----------|-----------|
| 100       | CSE       |
| 200       | Biology   |
| 300       | Biology   |

■ Note that  $X - Y = \emptyset \Leftrightarrow X \subseteq Y$

■ Note: Cannot write this query using = **all** and its variants

# (Self-Reading) Test if Unique

- **Unique** construct tests whether a subquery has any duplicate tuples in its result.
- **Unique** construct evaluates to “true” if a given subquery contains no duplicates.
- Find all courses that were offered at most once in 2009

```
select T.course_id
from course as T
where unique (select R.course_id
 from section as R
 where T.course_id= R.course_id
 and R.year = 2009);
```

Subquery: outside of where Clause

# Subqueries in the Form Clause

- SQL-92 allows a subquery expression to be used in **from** clause
- Find average instructors' salaries of those departments where the average salary is greater than \$42,000.

```
select dept_name, avg_salary
from (select dept_name,
 avg (salary) as avg_salary
 from instructor
 group by dept_name)
where avg_salary > 42000;
```

- Note above without **having** clause.
- Another way to write above query

```
select dept_name, avg_salary
from (select dept_name, avg (salary)
 from instructor
 group by dept_name) as dept_avg (dept_name, avg_salary)
where avg_salary > 42000;
```

| ID    | name       | dept_name  | salary |
|-------|------------|------------|--------|
| 22222 | Einstein   | Physics    | 95000  |
| 12121 | Wu         | Finance    | 90000  |
| 32343 | El Said    | History    | 60000  |
| 45565 | Katz       | Comp. Sci. | 75000  |
| 98345 | Kim        | Elec. Eng. | 80000  |
| 76766 | Crick      | Biology    | 72000  |
| 10101 | Srinivasan | Comp. Sci. | 65000  |
| 58583 | Califieri  | History    | 62000  |
| 83821 | Brandt     | Comp. Sci. | 92000  |
| 15151 | Mozart     | Music      | 40000  |
| 33456 | Gold       | Physics    | 87000  |
| 76543 | Singh      | Finance    | 80000  |

(a) The *instructor* table

# (Self-Reading) Scalar Subquery

- Scalar subquery is one which is used where a single value is expected
  - Runtime error if subquery returns more than one result tuple
- This below also a subquery, it is a scalar subquery
- List all departments along with the number of instructors in each department

```
select dept_name,
 (select count(*)
 from instructor
 where department.dept_name = instructor.dept_name)
 as num_instructors
from department;
```